Chapter 15

Networking

a network connection is just a file that someone else writes to

Networking is built in **layers**. Each one uses the layer below and hides its mess from the layer above: IP loses and reorders packets, TCP turns them into a reliable stream, HTTP gives the stream a request/response shape. Python has a module at each level:

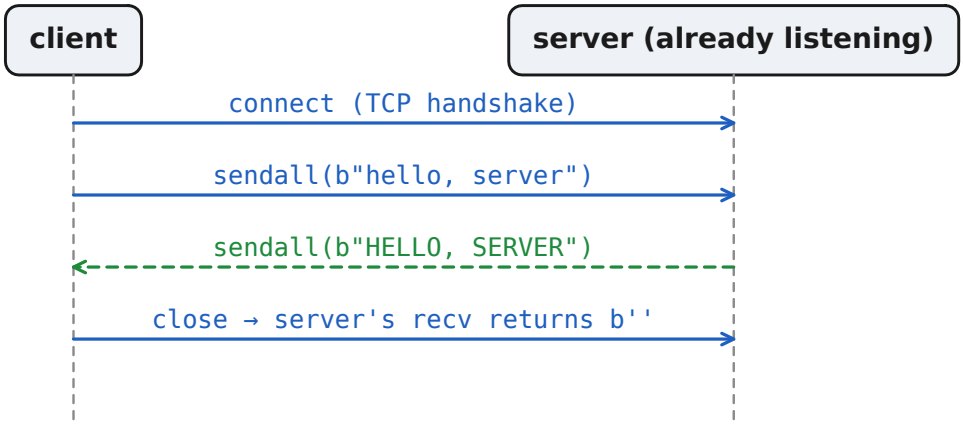
layer	what it gives the layer above	Python
application	HTTP, SMTP, your JSON	`urllib, http.client/server, smtplib`
security	TLS: encryption + identity	`ssl`
transport	TCP: reliable byte stream, ports	`socket (SOCK_STREAM)`
network	IP: addresses, packets, routing	`socket (AF_INET / AF_INET6)`
link	Ethernet, Wi-Fi	`the OS and the network card`

Every example in this chapter really runs: it starts a server on your own machine (`127.0.0.1`, "localhost") in a background thread and talks to it. No internet needed, and the output shown is what happened.

> **KEY IDEA**
> An address is a pair: **(host, port)**. The host finds the machine; the port (0-65535) finds the program on it. Ports below 1024 are reserved for well-known services (80 HTTP, 443 HTTPS, 22 SSH, 25 SMTP) and need admin rights to listen on.

TCP sockets: a phone line for bytes

A server **listens** on a port; a client **connects**; then both sides have a socket they can `sendall` bytes into and `recv` bytes out of, until one side closes:




```python
import socket, threading

def echo_server(listener):
    conn, addr = listener.accept()                  # wait for one client
    with conn:
        while data := conn.recv(1024):              # b"" means the client hung up
            conn.sendall(data.upper())

listener = socket.create_server(("127.0.0.1", 0))   # port 0: the OS picks a free one
port = listener.getsockname()[1]
threading.Thread(target=echo_server, args=(listener,), daemon=True).start()

with socket.create_connection(("127.0.0.1", port), timeout=5) as c:
    c.sendall(b"hello, server")
    print(c.recv(1024))
    print(c.getsockname()[0], "->", c.getpeername()[0], 1023 < port < 65536)
listener.close()
```

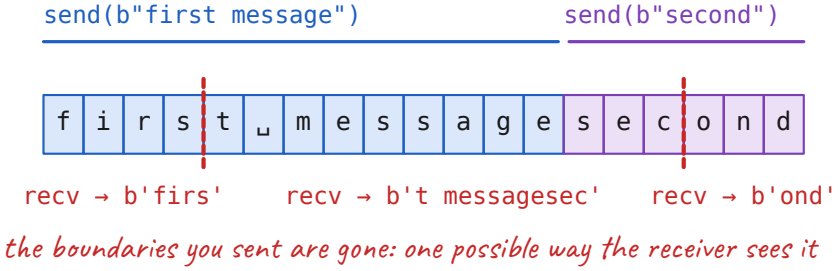
```
b'HELLO, SERVER'
127.0.0.1 -> 127.0.0.1 True
```

line 4: accept blocks

line 13: always a timeout

The big surprise: TCP has no messages

TCP promises the bytes arrive complete and in order, nothing more. Two `send`s can arrive as one `recv`, and one `send` can arrive as three:



the boundaries you sent are gone: one possible way the receiver sees it

```python
a, b = socket.socketpair()              # two connected sockets, handy for experiments
a.sendall(b"first message")
a.sendall(b"second")
print(b.recv(1024))                     # both sends, glued together

reader = b.makefile("rb")               # a file view of the socket
a.sendall(b"first message\n" + b"second\n")
print(reader.readline(), reader.readline())
```

```
b'first messagesecond'
b'first message\n' b'second\n'
```

line 7: framing: newline ends a message

RULE OF THUMB

Every protocol on TCP needs **framing**: a way to know where a message ends. The two classics are a delimiter (a newline, as SMTP, Redis and this chapter's practice 15.6 use) and a length prefix (send `struct.pack("!I", len(data))` first, as HTTP's Content-Length and practice 15.1 do). `recv(n)` returns **up to** n bytes, so loop until you have them all.

HTTP: text over TCP

HTTP is a framing convention on top of TCP: some lines of text, a blank line, then an optional body whose length is in a header. First, a small JSON API server using only the standard library:

```python
import json, threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

BOOKS = {1: {"id": 1, "title": "Dune"}, 2: {"id": 2, "title": "Emma"}}

class Api(BaseHTTPRequestHandler):
    def send_json(self, status, obj):
        body = json.dumps(obj).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()                          # the blank line
        self.wfile.write(body)

    def do_GET(self):                               # one method per HTTP verb
        if self.path == "/books":
            return self.send_json(200, list(BOOKS.values()))
        book_id = self.path.removeprefix("/books/")
        if book_id.isdigit() and int(book_id) in BOOKS:
            return self.send_json(200, BOOKS[int(book_id)])
        self.send_json(404, {"error": f"no such book: {self.path}"})

    def do_POST(self):
        data = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        book = {"id": max(BOOKS) + 1, "title": data["title"]}
        BOOKS[book["id"]] = book
        self.send_json(201, book)

    def log_message(self, *args):                   # keep the server quiet
        pass

server = ThreadingHTTPServer(("127.0.0.1", 0), Api)
threading.Thread(target=server.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{server.server_port}"
```

line 24: read exactly the body

line 32: a thread per request

This is what travels over the TCP connection. Each part has a job:

client sends

```
POST /books HTTP/1.1                    ← request line: method, path, version
Host: lib.example.org                   ← headers: Name: value
Content-Type: application/json
Content-Length: 17
↵                                       ← blank line (\r\n): headers end
{"title": "Ubik"}                       ← body: exactly Content-Length bytes
```

server answers

```
HTTP/1.1 201 Created                    ← status line: version, code, reason
Content-Type: application/json
Location: /books/3
↵
{"id": 3, "title": "Ubik"}
```

Now speak HTTP to our server by hand over a raw socket (dropping the `Date` header, which changes every run):

```
with socket.create_connection(("127.0.0.1", server.server_port), timeout=5) as c:
    c.sendall(b"GET /books/1 HTTP/1.0\r\nHost: localhost\r\n\r\n")
    reply = b""
    while chunk := c.recv(4096):                    # HTTP/1.0: server closes when done
        reply += chunk
print("\n".join(l for l in reply.decode().splitlines() if not l.startswith("Date")))
```

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.6
Content-Type: application/json
Content-Length: 26

{"id": 1, "title": "Dune"}
```

Calling a JSON API with urllib

`urllib.request` does the HTTP for you: building the request, reading the status and headers, following redirects. Errors with a status of 400 or more raise `HTTPError`, which still carries the response body:

```
import json, urllib.error, urllib.request

def get_json(url):
    with urllib.request.urlopen(url, timeout=5) as resp:
        return resp.status, json.load(resp)

print(get_json(base + "/books/2"))

req = urllib.request.Request(base + "/books", method="POST",
                             data=json.dumps({"title": "Ubik"}).encode(),
                             headers={"Content-Type": "application/json"})
with urllib.request.urlopen(req, timeout=5) as resp:
    print(resp.status, resp.headers["Content-Type"], json.load(resp))

try:
    get_json(base + "/books/99")
except urllib.error.HTTPError as e:
    print(e.code, e.reason, json.load(e))
server.shutdown()                                   # stop the background server
```

```
(200, {'id': 2, 'title': 'Emma'})
201 application/json {'id': 3, 'title': 'Ubik'}
404 Not Found {'error': 'no such book: /books/99'}
```

line 4: no timeout = may hang forever

line 18: the error body is JSON too

status	means	common ones
2xx	it worked	`200 OK`, `201 Created` (+ `Location`), `204 No Content`
3xx	look elsewhere	`301 Moved Permanently`, `304 Not Modified` (use your cache)
4xx	**you** got it wrong: don't retry as is	`400`, `401` who are you?, `403` not allowed, `404`, `409` conflict, `429` slow down
5xx	**the server** broke: retry later	`500`, `502 Bad Gateway`, `503 Unavailable`

method	does	safe?	repeatable (idempotent)?
`GET`	read	**yes**	**yes**
`POST`	create / do something	no	**no**: twice makes two
`PUT`	replace at this URL	no	**yes**
`DELETE`	remove	no	**yes**

WHY

Idempotency decides what is safe to **retry** after a timeout: you don't know whether the request arrived. Retrying a GET or PUT is harmless; retrying a POST might lend the book twice, which is why payment APIs make you send an idempotency key.

URLs, TLS, and email

```python
from urllib.parse import parse_qs, quote, urlencode, urljoin, urlsplit

u = urlsplit("https://lib.example.org:8443/search?q=dune&page=2#results")
print(u.scheme, u.hostname, u.port, u.path, u.query, u.fragment)
print(parse_qs(u.query))
print(urlencode({"q": "science fiction", "lang": "en"}), quote("a/b c&d", safe=""))
print(urljoin("https://lib.example.org/books/", "../authors/7"))
```

```
https lib.example.org 8443 /search q=dune&page=2 results
{'q': ['dune'], 'page': ['2']}
q=science+fiction&lang=en a%2Fb%20c%26d
https://lib.example.org/authors/7
```

line 6: never glue URLs with +

TLS (the S in HTTPS) encrypts the stream and proves the server's identity with a certificate. `urllib` uses it automatically for `https://`. The one rule is to leave verification on:

```python
import ssl

ctx = ssl.create_default_context()          # trusted CAs, hostname checks, modern TLS
print(ctx.verify_mode == ssl.CERT_REQUIRED, ctx.check_hostname, ctx.minimum_version)
```

```
True True 771
```

wrapping a socket yourself (not run here: needs the internet)

```python
with socket.create_connection(("example.org", 443), timeout=10) as raw:
    with ctx.wrap_socket(raw, server_hostname="example.org") as tls:
        print(tls.version(), tls.getpeercert()["subject"])
```

TRAP

`ssl._create_unverified_context()`, `verify=False` in requests, or `CERT_NONE` make the error go away by letting anyone in the middle read and change the traffic. Fix the certificate (or install the right CA) instead. And if your server fetches URLs that users supply, check where they point first: `http://169.254.169.254/` is how cloud credentials get stolen (SSRF).

Email: build with email, send with smtplib

```python
from email.message import EmailMessage

msg = EmailMessage()
msg["From"], msg["To"] = "library@example.org", "ana@example.org"
msg["Subject"] = "Your loan is due"
msg.set_content("Dune is due back on 12 October.\n")
print(msg.as_string())
```

```
From: library@example.org
To: ana@example.org
Subject: Your loan is due
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0

Dune is due back on 12 October.
```

sending (not run here: needs a mail server)

```python
import smtplib

with smtplib.SMTP("smtp.example.org", 587, timeout=10) as s:
    s.starttls(context=ssl.create_default_context())    # upgrade to TLS first
    s.login(user, password)                              # from the environment (Ch 14)
    s.send_message(msg)
```

need	use
a quick GET/POST from a script	`urllib.request` (stdlib)
many requests to one host, low-level control	`http.client` (keeps the connection open)
a real application: sessions, retries, nicer API	`requests` or `httpx` from pip (Ch 19)
serve HTTP in production	a framework (Flask, Django, FastAPI) behind a real server, not `http.server`

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 15.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 15.1` shows other valid solutions and their trade-offs.

15.1 build from a spec Length-prefixed messages over TCP
15.2 fix the bug Parse a raw HTTP request properly
15.3 learn an unseen module URL surgery with urllib.parse
15.4 build from a spec A JSON API client that handles failure
15.5 build from a spec Serve a small JSON API
15.6 mini-project A line-based key-value server

PASSPORT STAMP ask the same question of every new language (Appendix B)

10. I/O

Network I/O is bytes over sockets: `socket` (TCP byte stream, needs framing), `ssl` for TLS, `urllib.request` / `http.client` for HTTP, `http.server` for tiny servers.

12. Stdlib and tooling

`socket`, `socketserver`, `ssl`, `urllib.request`, `urllib.parse`, `http.client`, `http.server`, `http.HTTPStatus`, `json`, `email`, `smtplib`.

IN OTHER LANGUAGES

The socket API is the same everywhere because it's the OS's (BSD sockets, 1983): C `socket()/bind()/listen()/accept()`, Java `ServerSocket`, Go `net.Listen`, Rust `std::net::TcpListener`. HTTP clients differ in style, not substance: Java `HttpClient`, Go `net/http`, JS `fetch`.

DEEP DIVE (your vault)

Vault notes: 18.2 Networks and layered agreements · 18.3 Addresses, ports, and names · 18.4 TCP and the meaning of a byte stream · 18.5 TLS - authenticated protected transport · 18.7 HTTP messages · 18.8 Methods and method properties · 18.9 Status codes · 18.13 From a request to a Python callable · 18.26 Server-side request forgery.